

Formal Methods in Software Engineering, 2026

Assignment on Java PathFinder

Due date: April 16th, 2026, 11.59 pm. Please upload into the Teams Assignment a single zip file that contains all the files asked for below. Any delayed submissions or submissions via other mechanisms such as email or chat will not be accepted.

Problem 1. Consider the skeleton code below:

```
import gov.nasa.jpf.symbc.Debug;

public class FindMin {
    public static int findMin(int[] arr) {
        // find the largest element in arr and return it
    }

    public static void main(String[] args) {
        int[] a = new int[2];
        ar[0] = Debug.makeSymbolicInteger("m0");
        ar[1] = Debug.makeSymbolicInteger("m1");
        int r = findMin(ar);

        // assertion
        // write code here that will check if r is the minimum
        // value in array 'ar' and invoke 'assert false' otherwise.
    }
}
```

Put this code into a file *FindMin.java*. Create a file *FindMin.jpf*, setting the option *symbolic.method* to the *findMin* method above, with the parameter **arr** being concrete. Note that although we are passing the array concretely, we are populating it with (integer typed) symbols. Your code, both within the *findMin* function and in the assertion, should not assume that the array has two elements only. The configuration parameters in your *FindMin.jpf* can be similar to *Branches.jpf*, which has been shared with you as part of the class materials.

Run Symbolic Path Finder. Draw a tree that represents the entire symbolic execution, in the same format as shown in Slide 33 in the class slides. Be sure to mention the source line number and choice point ID (as shown in the “**forward**” lines in the tool’s output) at each choice point node. You may hand-draw the tree, but do it neatly with no scratching out, etc. Show the final path condition and any one concrete input that is a solution to this path condition at the end of each path in the tree. Put this drawing in a file called *findmin-tree.pdf*.

Submit the .java file, the .jpf file, the file findmin-tree.pdf, and a text file named *findmin-tree.out* containing the entire output from the JPF run.

Problem 2. Consider the method below:

```
1: public static boolean hasCycleBounded(NodeSimple head, int maxSteps) {
2:     NodeSimple slow = head;
3:     NodeSimple fast = head;
4:     int steps = 0;

5:     while (fast != null && fast.next != null && steps < maxSteps) {
6:         slow = slow.next;
7:         NodeSimple t = fast.next;
8:         fast = t.next;
9:         if (slow == fast)
10:            return true;
11:         steps++;
12:     }
13:     return false;
14: }
```

`NodeSimple` is a class with a `next` field. Place a call to the method above from `main()`, passing some arbitrary object to the method and the constant 1. Put the code above in a file *NodeSimple.java*, and create a file *NodeSimple.jpf*, setting the option *symbolic.method* to the method above, with the parameter `head` being symbolic and the parameter `maxSteps` being concrete. The rest of the options can be similar to the ones in the *NodeSimple.jpf* that we used in class. Run Symbolic Path Finder.

1. Draw a tree that represents the entire symbolic execution, in the same format as shown in Slides 47, 49, and 51 in the class slides. Be sure to include the source line number (as shown in the code listing above) along with the choice point ID in each choice point node. You may hand-draw the tree, but do it neatly with no scratching out, etc. You may use more than one page to draw the tree in case one page does not suffice.
2. Corresponding to every “end” node (i.e., leaf node) in the tree, show (i.e., draw) a concrete linked list that satisfies the path condition of the path that ends at that node. To avoid clutter, you can show this part after the tree.

Combine the solutions to the previous part and this part into a single file called *node-tree.pdf*.

Submit the .java file, the .jpf file, the file node-tree.pdf, and a text file named *node-tree.out* containing the entire output from the JPF run.